

DAAB

System Architecture

The data & knowledge backbone: a knowledge-graph platform that connects to an organisation's live databases and documents and serves them to AI agents through the Model Context Protocol (MCP) and to people through a web dashboard.



VERSION
1.0

DATE
2026-09-24

CODE BASELINE
go @ 16dc548 · python @ 6966d4b · web @
907bc8b

Contents

1 Overview & design principles

What the platform does · Design principles · Technology stack

2 System context

External parties · Where data goes

3 Containers & components

Container view · Inside the Go API server

4 Knowledge graph model

Node types · Edges and the whitelist · Write gates and history

5 Data model

Graph core · Access · Data sources · Queries and writes · Documents · Resolution and memory

6 Ingestion pipelines

Documents · Live databases · Source code

7 Retrieval: search, hybrid ranking & GraphRAG

Search modes · Graph navigation · GraphRAG · Vector indexes

8 Natural-language data query

Agent query pipeline · Execution guards · Dashboard conversational analysis

9 Controlled writes

Single-row insert · Write playbooks

10 Entity resolution & master records

Extraction · Resolution stages · Applying merges · Master and derived records

11 Agent memory

Memory write · Hybrid recall · Upkeep

12 MCP bridge

Transports · Authentication modes · Per-call pipeline · Tool catalogue

13 Web dashboard

Page map · BFF proxy and sessions · Real-time · Graph visualisation

14 Inter-service communication

Job lists · Synchronous calls · Delivery semantics

A Appendix A – MCP tool catalogue

Every tool with its class and purpose

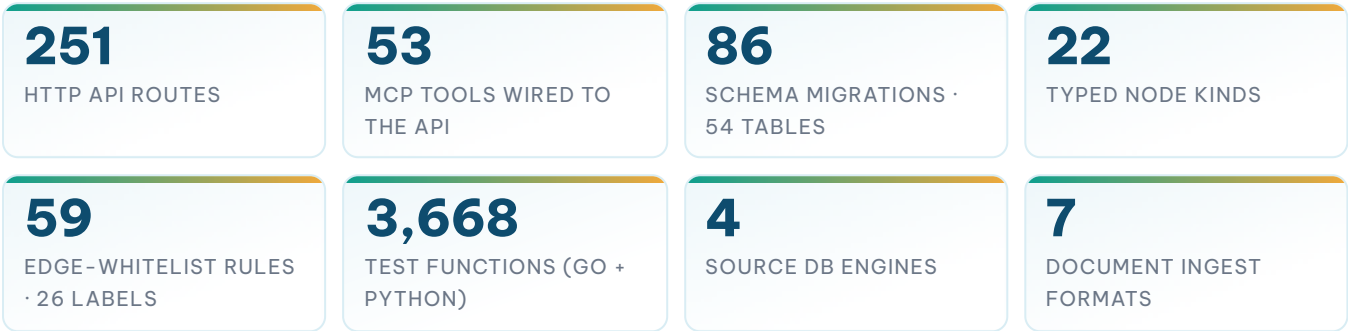
B Appendix B – REST API summary

Route counts by area · Representative endpoints

Status vocabulary used throughout: **LIVE** implemented, wired and exercised with real data · **BUILT** implemented and wired, not yet exercised with production data · **PLANNED** roadmap, flag-off or stub. All figures in this document were measured against the code baseline printed on the cover.

1 Overview & design principles

DAAB turns an organisation's operational databases and document estate into one queryable knowledge graph. AI agents reach it through a standards-based MCP server (55 tools advertised, 53 of them wired to the API); people reach it through a web dashboard. Underneath is a single PostgreSQL system of record that holds the graph, its version history, full-text indexes and vector embeddings together.



Method: routes counted from registered `HandleFunc` patterns (247 in the handler package + 4 on the outer mux); MCP tools = 55 advertised schemas minus 2 without a server route yet; tables are distinct `CREATE TABLE` names across migrations 000001–000087, cross-checked against a migrated database; tests = 2,753 Go `func Test*` + 915 Python `def test_*` functions (pytest collects 943 Python tests, counting parametrised cases). Source engines: PostgreSQL, MySQL, MariaDB, Microsoft SQL Server. Formats: PDF, DOCX, Markdown, TXT, CSV, XLSX, JSON.

What the platform does

- **Connects to live databases.** Registers customer PostgreSQL, MySQL, MariaDB and SQL Server sources, extracts their schema, and generates a graph of tables, foreign keys and AI-detected implicit relationships.
- **Ingests documents.** Parses, OCRs (including Vietnamese), deduplicates, sections, chunks and embeds documents, then extracts entities and relationships into the same graph.
- **Answers questions over data.** For agents, translates a natural-language question into a validated *query plan*, renders parameterised SQL deterministically and runs it read-only against the source; the dashboard chat adds a conversational mode (Chapter 8).
- **Retrieves knowledge.** Offers full-text, semantic and hybrid (Reciprocal Rank Fusion) search, passage-level retrieval with citations, multi-hop traversal and cross-document GraphRAG.
- **Resolves entities.** Merges duplicate people, organisations and concepts through a staged, confidence-gated and reversible pipeline.
- **Remembers.** Gives agents a durable, scoped memory (`remember` / `recall`) with hybrid recall.
- **Writes back, under control.** Opt-in single-row inserts and pre-approved multi-step write playbooks, all audited.

Design principles

Principle	How it shows up in the code
One transactional system of record	Graph nodes, edges, version history, full-text vectors (<code>tsvector</code>) and embeddings (<code>pgvector</code>) all live in PostgreSQL 16. No separate graph or vector database to synchronise; a write and its index update commit together.

Principle	How it shows up in the code
Single write path	The Go API server owns the database. Graph writes from Python workers, the dashboard and the MCP bridge go through its REST API, so validation, authorisation and audit apply to every writer. The one exception is an optional direct database connection for the worker (set in the development stack) that marks merge suggestions for review and looks up documents.
Schema as configuration	Node types, required properties, completeness rules and the edge whitelist are declared in <code>config.yaml</code> and enforced by validation gates. New knowledge kinds need configuration, not new tables.
Versioned history	Every node update is copied to a versions table by a database trigger, and graph edits never remove versions. Knowledge nodes are deprecated or superseded rather than deleted; the exception is document regeneration, which replaces a changed document's sections and chunks (and their version rows). Updates use optimistic locking.
The model proposes, code decides	For agent (MCP / API) data queries the LLM returns a structured query plan, never raw SQL; the dashboard's agentic chat lets the model write SELECT statements, checked by a read-only validator with row caps. Merge decisions pass rule, reranker and confidence gates. Write playbooks are proposed deterministically and must be approved before they run.
Local embeddings	Embeddings (multilingual-e5-small, 384 dimensions) and the reranker (bge-reranker-v2-m3) run on CPU inside the platform. Search and resolution carry no per-call embedding cost, and document text is not sent out to be embedded (an optional hosted embedding service is used only for table selection, when configured).
Provider-agnostic AI	LLM providers are database-registered with encrypted keys, selected by priority with automatic failover and a circuit breaker. Models are assigned per function (e.g. query planning vs extraction). Dashboard chat and, when configured, chunk-level extraction call providers from the Python services (Chapter 3).
Standard library first	The Go server uses <code>net/http</code> and <code>database/sql</code> with hand-written SQL, with no web framework and no ORM. That gives it a small dependency surface and predictable queries.

Deployment status: pre-production

DAAB runs today as a self-hosted Docker Compose stack for development and demonstrations **LIVE**. A release bundle (v1.0.0) and a cloud development environment on AWS (ECS, RDS, ElastiCache) with CI/CD for the Go service are built **BUILT**; production deployment is **PLANNED**. No production customers, SLAs or scale figures are claimed in this document. Infrastructure, security and operations are covered in Document 03.

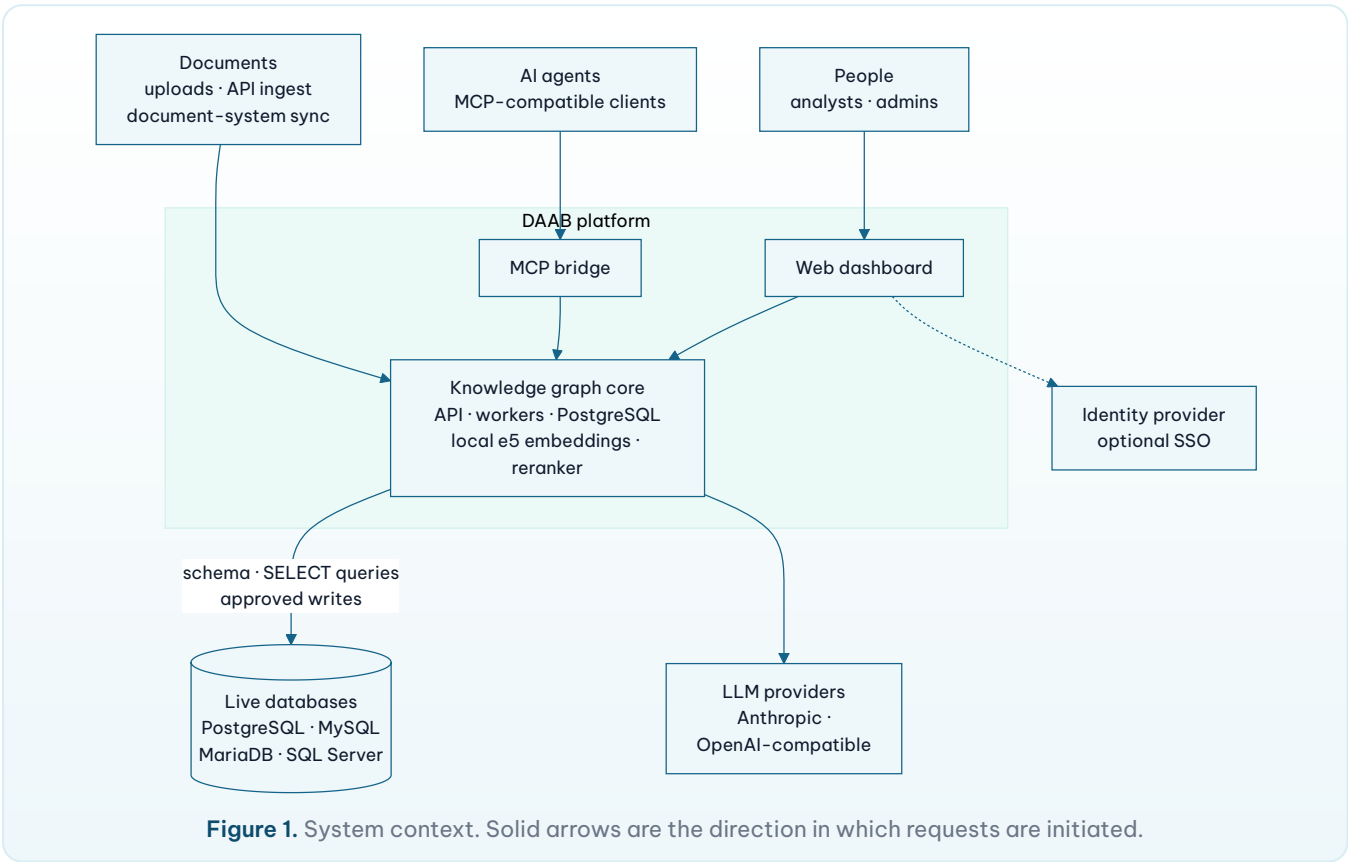
Technology stack

Layer	Technology
API server & MCP bridge	Go 1.25, stdlib <code>net/http</code> (1.22 method patterns), <code>database/sql</code> + <code>lib/pq</code> , <code>golang-migrate</code> , official MCP Go SDK, <code>log/slog</code> , AES-256-GCM
Workers & AI services	Python 3.12, FastAPI + uvicorn, httpx, pydantic-settings, sentence-transformers (CPU), tree-sitter, Tesseract + RapidOCR, pypdf/pypdfium2, asyncpg / pymysql / pymssql

Layer	Technology
Web dashboard	Next.js 16 (App Router), React 19, TypeScript strict, TanStack Query 5, iron-session, Tailwind 4, Three.js / React Three Fiber, Cytoscape
Data	PostgreSQL 16 with <code>pgvector</code> , <code>pg_trgm</code> , <code>unaccent</code> ; Redis 7 (job lists, short-lived agent state)
AI providers	Anthropic API, Claude subscription via OAuth, any OpenAI-compatible endpoint; local e5 embeddings and bge reranker

2 System context

DAAB sits between an organisation's data (databases and documents) and the consumers of that knowledge: people in a browser and AI agents speaking MCP.



Where data goes

Embeddings and reranking run locally. Text leaves the deployment only when it is sent to the LLM provider an administrator has configured, or to an optional hosted embedding service for table selection, when configured. Source-database credentials and provider API keys are stored encrypted (AES-256-GCM) and decrypted only when used. For dashboard chat, the Go server passes the provider key and the still-encrypted connection string to the Python indexer with each request; the indexer decrypts the connection string itself and queries the source database directly (Chapter 8).

External party	Interface	What flows
People	Web dashboard (HTTPS)	Graph exploration, conversational data analysis, source management, ingestion review, administration.

External party	Interface	What flows
AI agents	MCP over stdio or Streamable HTTP	53 tools wired to the API: search, traversal, document navigation, data query, memory, ingestion, controlled writes.
Live databases	Native drivers per engine	Schema catalogue extraction; SELECT-only query execution; opt-in, audited inserts and playbook transactions.
Document sources	File upload, public ingest API, document-management-system sync (configured as a source connection)	Files and structured drafts, which become documents, sections, chunks and entities.
LLM providers	HTTPS (provider APIs)	Query planning, structured extraction, relationship detection, merge verification, answer composition.
Identity provider	JWT verification (JWKS)	Optional single sign-on. A verified login is exchanged for a DAAB API key.

3 Containers & components

Seven containers run from three images: a Go image (API server and MCP bridge), a Python image (indexer service and worker) and a Next.js image (dashboard), plus PostgreSQL and Redis.

Container	Technology	Responsibility
kg-server	Go, stdlib HTTP	System of record and API gateway. Owns the graph, validation gates, authN/Z, project RBAC, audit. Hosts the natural-language data-query pipeline, the live-DB schema extraction and graph generation, the write path, the AI provider layer, GraphRAG link building and the in-process job engine.
kg-bridge	Go, MCP SDK	Exposes the REST API as MCP tools over stdio or Streamable HTTP, with its own auth, rate limiting and project scoping (Chapter 12).
kg-migrate	Go, golang-migrate	Applies the 86 up/down SQL migrations.
indexer	Python, FastAPI	Serves query and passage embeddings (e5, 384-d) and the dashboard's streaming chat engines (fast path and agentic path, Chapter 8). Also operator endpoints for re-embedding and backfill.
worker	Python, asyncio	Consumes four Redis job lists carrying five kinds of work: document ingestion and OCR, document-system sync (these two share one list), code indexing, chunk-level entity extraction and resolution, and memory embedding.
dashboard	Next.js 16, React 19	Web UI and backend-for-frontend (BFF) proxy. The browser never calls the Go API directly (Chapter 13).
postgres / redis	PostgreSQL 16, Redis 7	Durable state / job lists and short-lived agent clarification state.

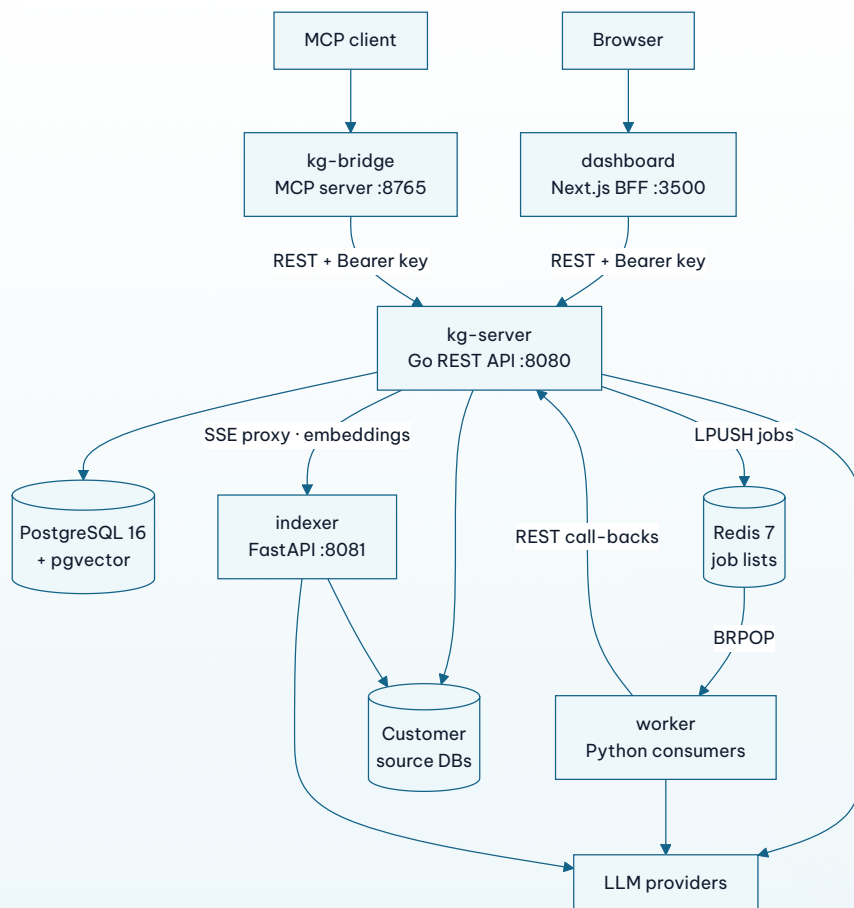


Figure 2. Container view. `kg-migrate`, a third Go binary, is the schema migration CLI.

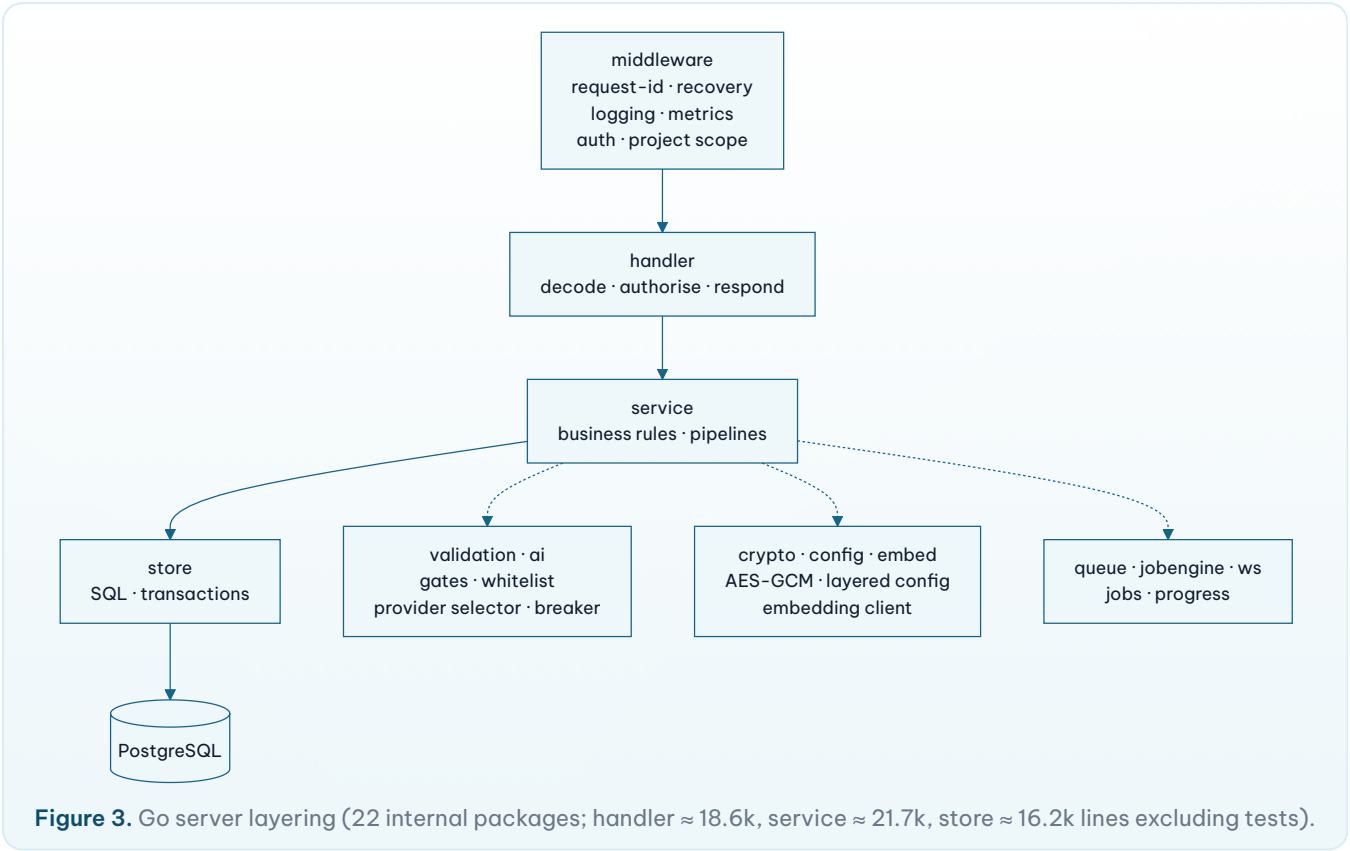
■ Inside the Go API server

The server is a classic three-layer design wired by constructor injection in a single composition root. Handlers decode requests and authorise; services hold business rules, validation gates and pipelines; stores hold hand-written SQL and transaction helpers.

Request pipeline. Every request passes, outer to inner: request-ID → panic recovery → structured access logging → metrics; routes under `/api/` then pass API-key authentication and project resolution before reaching a handler. Project scope is enforced by that middleware and on project-scoped handlers; handlers that receive a project in the request body re-check access against the caller's key, and writes by user-linked keys require a project role (viewer < developer < admin). Applying per-project checks uniformly to every by-ID read and to keys not linked to a user is a hardening item before production (Document 03, Chapter 8). A project that has been archived rejects every mutating method.

AI provider layer. Providers (Anthropic API, Claude subscription via OAuth, OpenAI-compatible endpoints) are registered in the database with encrypted keys and a priority. The selector picks the highest-priority healthy provider, fails over on error, and wraps each provider in a circuit breaker (opens after 5 failures within 10 minutes, half-opens after 30 seconds). A per-function resolver lets administrators assign different models to different jobs. Every call routed through this layer is logged with its token usage and cost for budgeting. Two paths call providers from the Python services instead: dashboard chat, which receives the selected provider's key from the Go server per request, and chunk-level extraction when it is configured with its own endpoint; neither passes through this layer's circuit breaker or usage log.

Configuration. The server reads configuration in four layers, each overriding the one before: `config.yaml` (the knowledge schema, edge whitelist, completeness, search, AI, cost and resolution settings), then a per-environment YAML file (development, staging, production, self-hosted), then `KG_*` environment variables, then a `system_settings` table for runtime overrides and feature flags.



4 Knowledge graph model

Every piece of knowledge is a typed node in a single table, with type-specific properties in JSONB and a schema per type declared in configuration. Edges are typed and must match a whitelist of source → relationship → target rules.

Node types

The database enforces 22 node types through a CHECK constraint; property schemas, required fields and completeness rules for the content types live in `config.yaml`. The types fall into four families:

Family	Types	Produced by
Project knowledge	<code>decision</code> <code>concept</code> <code>requirement</code> <code>task</code> <code>architecture</code> <code>discovery</code> <code>initiative</code> <code>session</code>	Agents and people via MCP / API; live-DB schema generation (tables become <code>architecture</code> nodes); code indexing
Documents	<code>document</code> <code>document_section</code> <code>document_chunk</code> <code>dataset</code> <code>external</code> <code>document_ref</code>	Document ingestion pipeline

Family	Types	Produced by
Entities	person organization event location artifact project	Chunk-level LLM extraction (together with concept)
Records	master_record derived_record	External systems write back derived_records (idempotent per source record); master_record is reserved for resolved entities

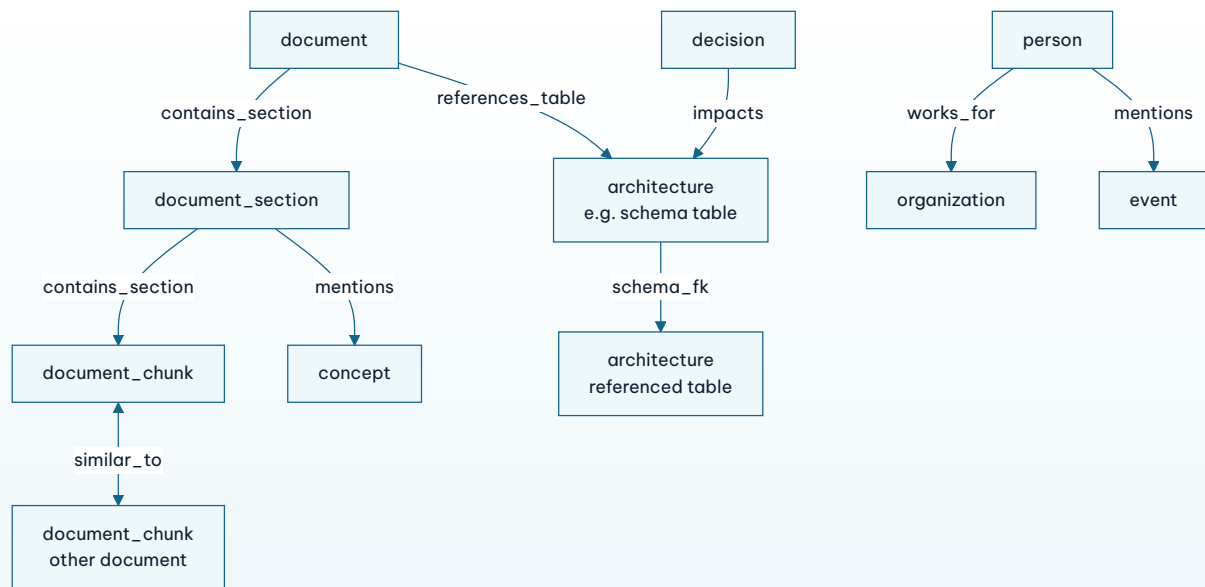


Figure 4. Representative node and edge types across the document, entity, knowledge and schema families.

Edges and the whitelist

Edges carry a type, JSONB properties (for example confidence and reasoning on AI-detected relations), provenance (creator, session) and a uniqueness constraint on (source, target, type). The edge type is deliberately *not* constrained in the database: the whitelist in configuration is the single source of truth. It holds **59 rules over 26 relationship labels**, each with an optional cross-project permission.

Group	Labels
Structure	contains_section part_of upload_batch
Semantics	mentions related_to relates_to about evidence derived_from similar_to works_for causes
Engineering knowledge	implements impacts depends_on blocked_by fulfilled_by supersedes references_component
Data & cross-source	schema_fk schema_implicit schema_many_to_many references_table data_maps_to_table cross_source_reference references_document

Write gates and history

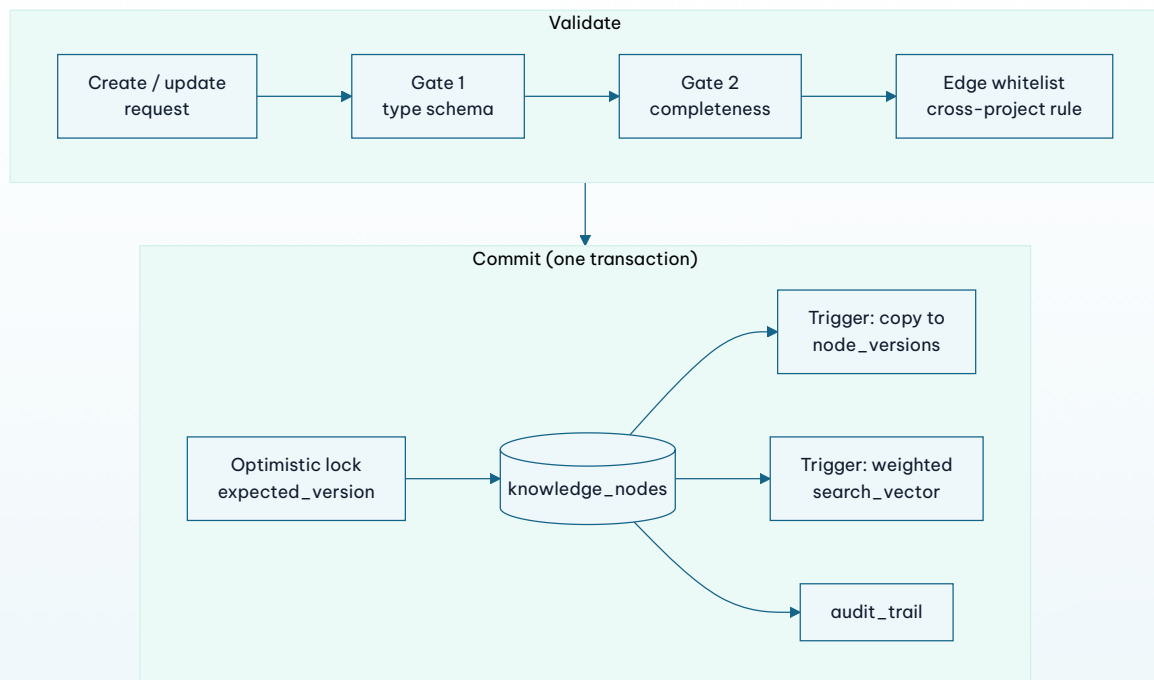


Figure 5. Every graph write passes schema and completeness gates, whitelist checks and optimistic locking; history and search indexes are maintained by database triggers in the same transaction.

- **Gate 1** validates properties against the type's schema; **Gate 2** applies completeness rules and profiles (for example, a decision needs a rationale).
- **Optimistic locking:** a partial update may carry `expected_version` ; a mismatch returns HTTP 409 instead of overwriting a concurrent change.
- **Version history:** an update trigger copies the prior row into `knowledge_node_versions` (unique on node + version). Nodes move to `deprecated` or `superseded` ; there is no delete on the node API. When a changed document is regenerated, its old sections and chunks, with their version rows, are removed and rebuilt (Chapter 6).
- **Search vector:** a trigger maintains a weighted `tsvector` (title and name weighted highest, then descriptive, then contextual fields), plus a trigram index on titles.

5 Data model

The schema is managed by 86 forward/backward migration pairs and holds 54 application tables. Three PostgreSQL extensions do the heavy lifting: `pgvector` (vector similarity), `pg_trgm` (fuzzy title match) and `unaccent` (accent-insensitive Vietnamese search). The diagrams below split the schema by domain; only key tables and columns are shown.

5.1 Graph core

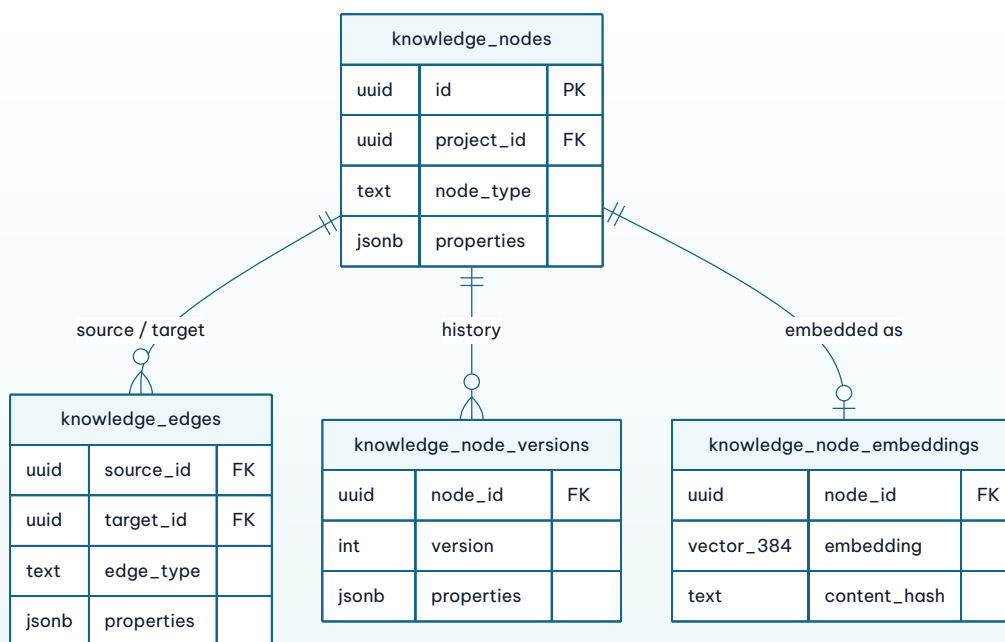


Figure 6. Graph core: a single typed-node table with JSONB properties, whitelisted edges, trigger-maintained history and 384-dimension embeddings.

5.2 Identity, access and AI administration

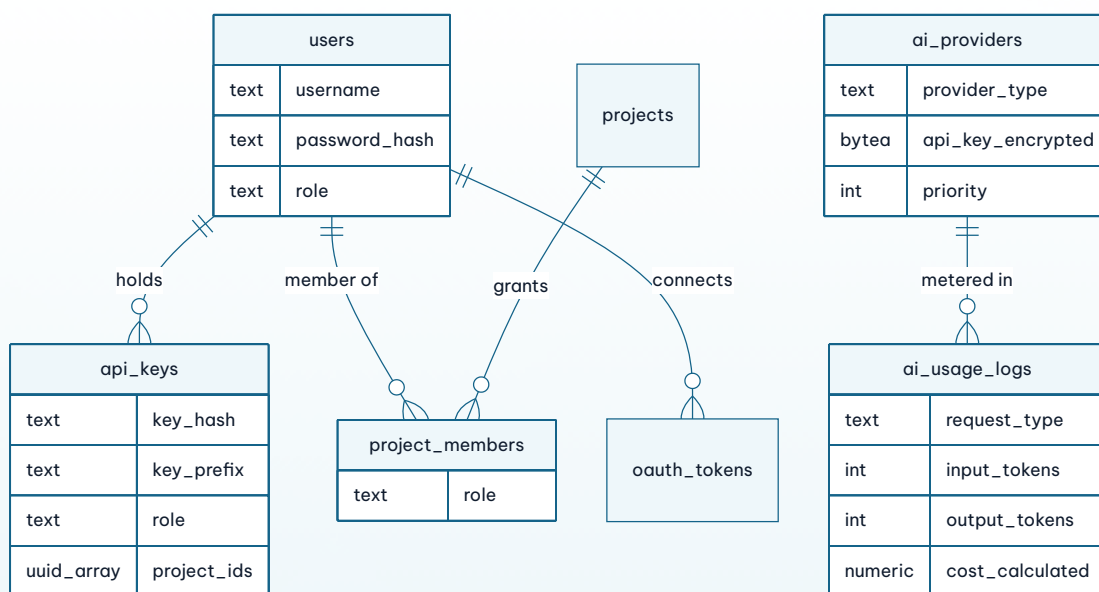


Figure 7. Access domain: API keys are stored only as hashes; provider keys and OAuth tokens are encrypted; AI calls routed through the Go server are metered.

5.3 Live data sources and schema catalogue

- **Soft delete and test status.** Data sources are soft-deleted and record the time and outcome of their last connection test; connection strings are stored only as AES-256-GCM ciphertext.
- **Human overrides.** Tables and columns carry both an AI-generated description and a separate user description, so curated knowledge is never overwritten by regeneration.
- **Job tracking.** `sync_jobs` tracks progress for schema extraction, schema sync, schema description and graph generation (pending → running → completed / failed / cancelled), with percentages that feed the dashboard's live progress stream. Graph runs also record their node and explicit, implicit and rejected edge counts in `kg_generation_jobs`.

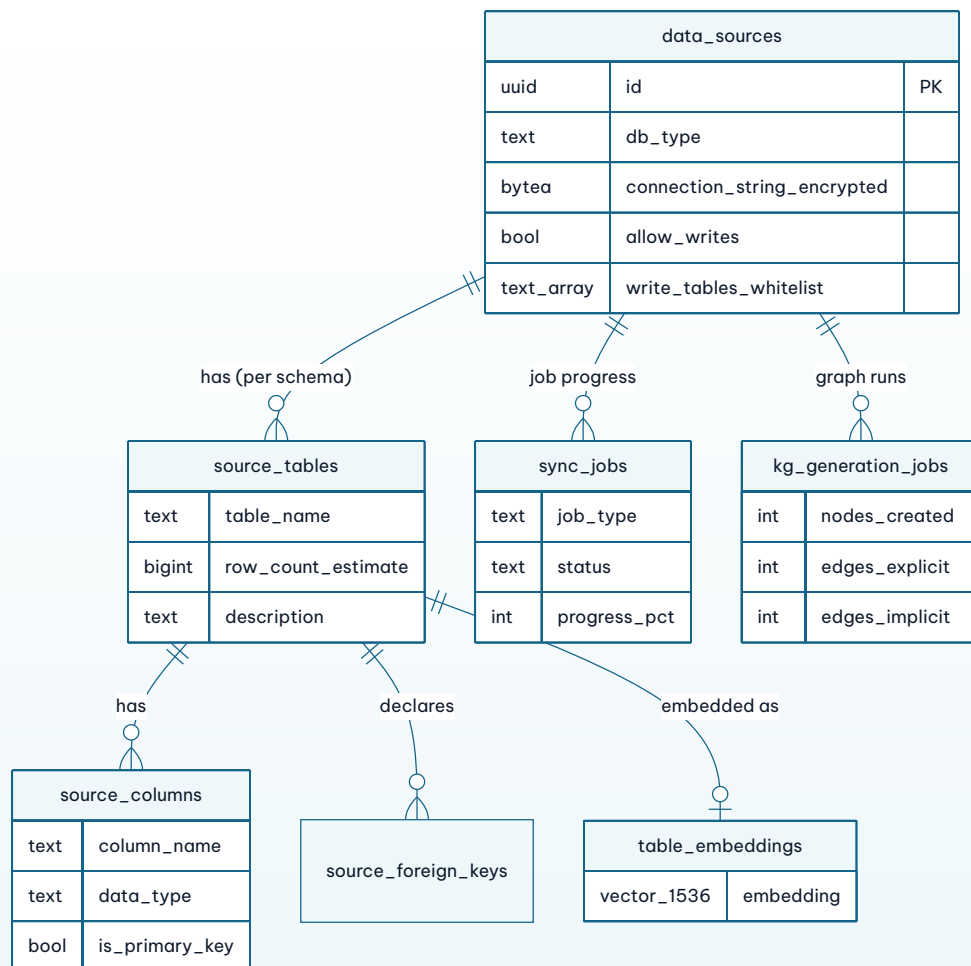


Figure 8. Data-source domain: encrypted connection details, the extracted schema catalogue, and the job tables that drive sync and graph generation.

5.4 Queries, conversations and writes

- **Reproducible answers.** Every query stores the question, the SQL that was generated and the result, so any answer can be traced back to the exact statement that produced it.
- **Plan cache keyed on schema.** Cached plans are keyed by normalised question and schema fingerprint and count their hits; a schema change produces a new fingerprint rather than a stale plan.
- **Playbooks are unique per data source by name,** versioned, and record who approved them and when, plus the reason a playbook was marked stale.
- **Write audit captures inputs.** Each attempt records actor, operation, target table, input values and outcome, whether it was successful, rejected by a gate, or failed at the database.
- **Conversations** keep per-message token counts, latency and model, and link to the query they ran; chat clarifications are held in their own table with an expiry.

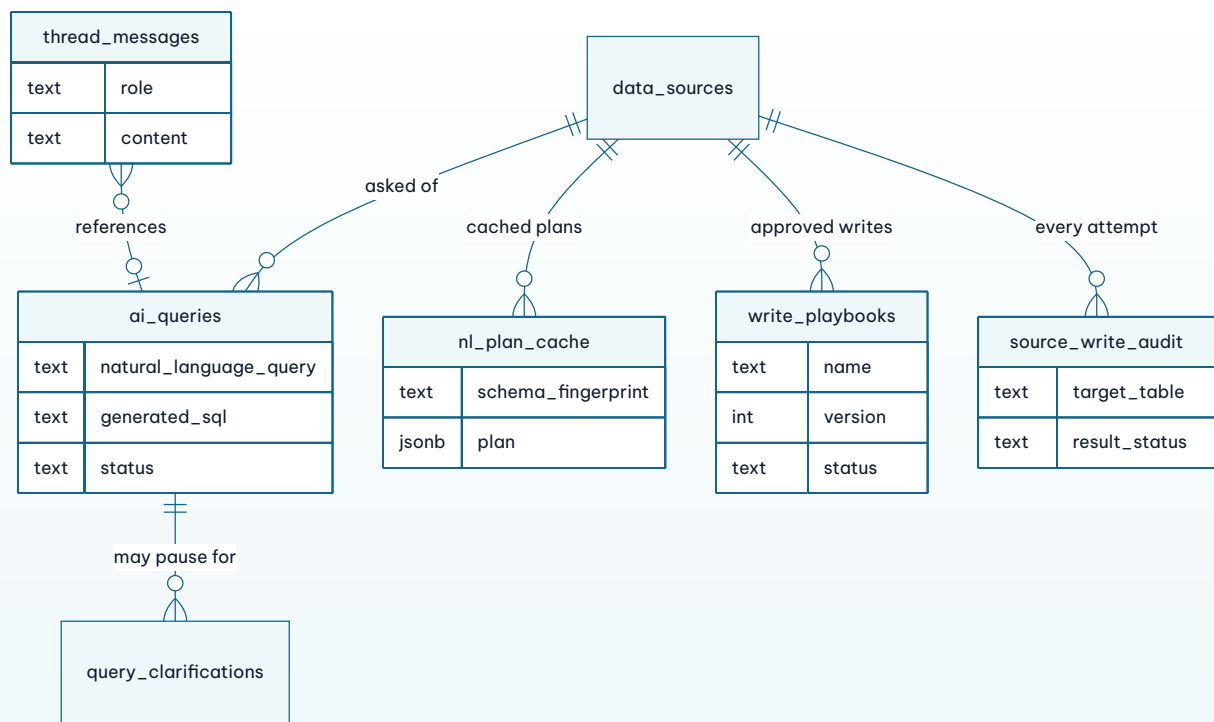


Figure 9. Query and write domain: every natural-language query, cached plan, playbook and write attempt is persisted.

5.5 Documents and ingestion

- **One draft per source item.** Drafts are unique on (project, source type, source ID), so re-submitting the same item updates it instead of duplicating it; each draft links to the graph node it produced and records who approved it.
- **One canonical record per draft,** carrying the SHA-256 content hash used by the three deduplication tiers. Canonical records and uploaded files are soft-deleted, which lets a document revert to an earlier version cleanly.
- **Source connections** (not drawn) hold per-project connector configuration and status, with any credentials held encrypted.

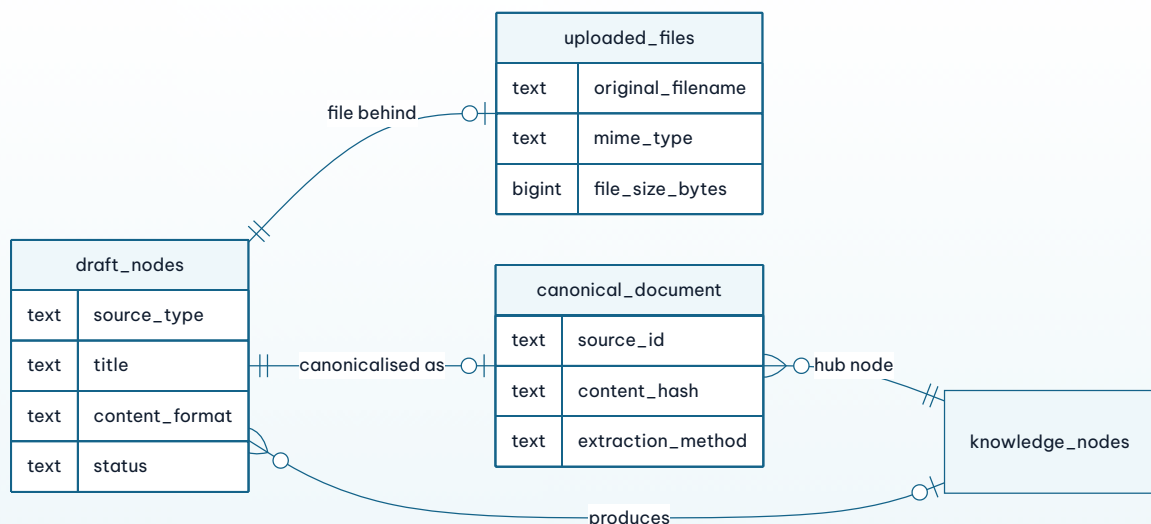
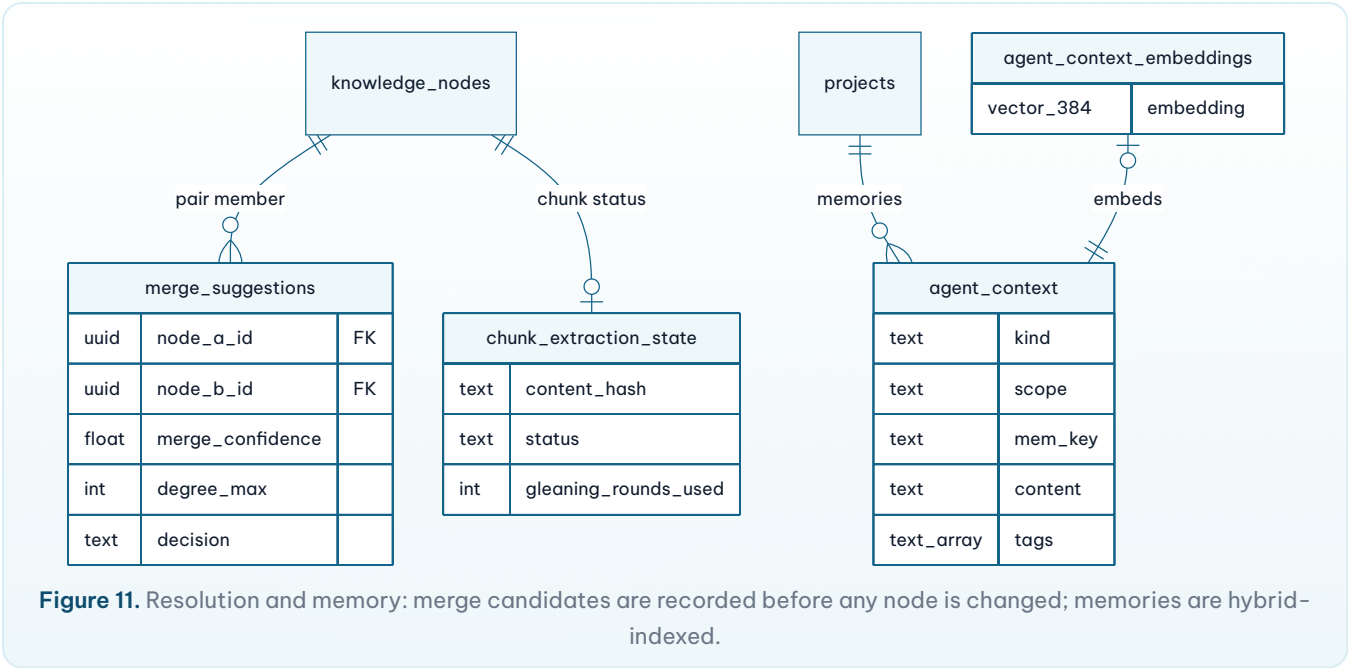


Figure 10. Ingestion domain: every input becomes a draft; canonical documents carry the content hash used for "ingest once" deduplication.

5.6 Entity resolution and agent memory



6 Ingestion pipelines

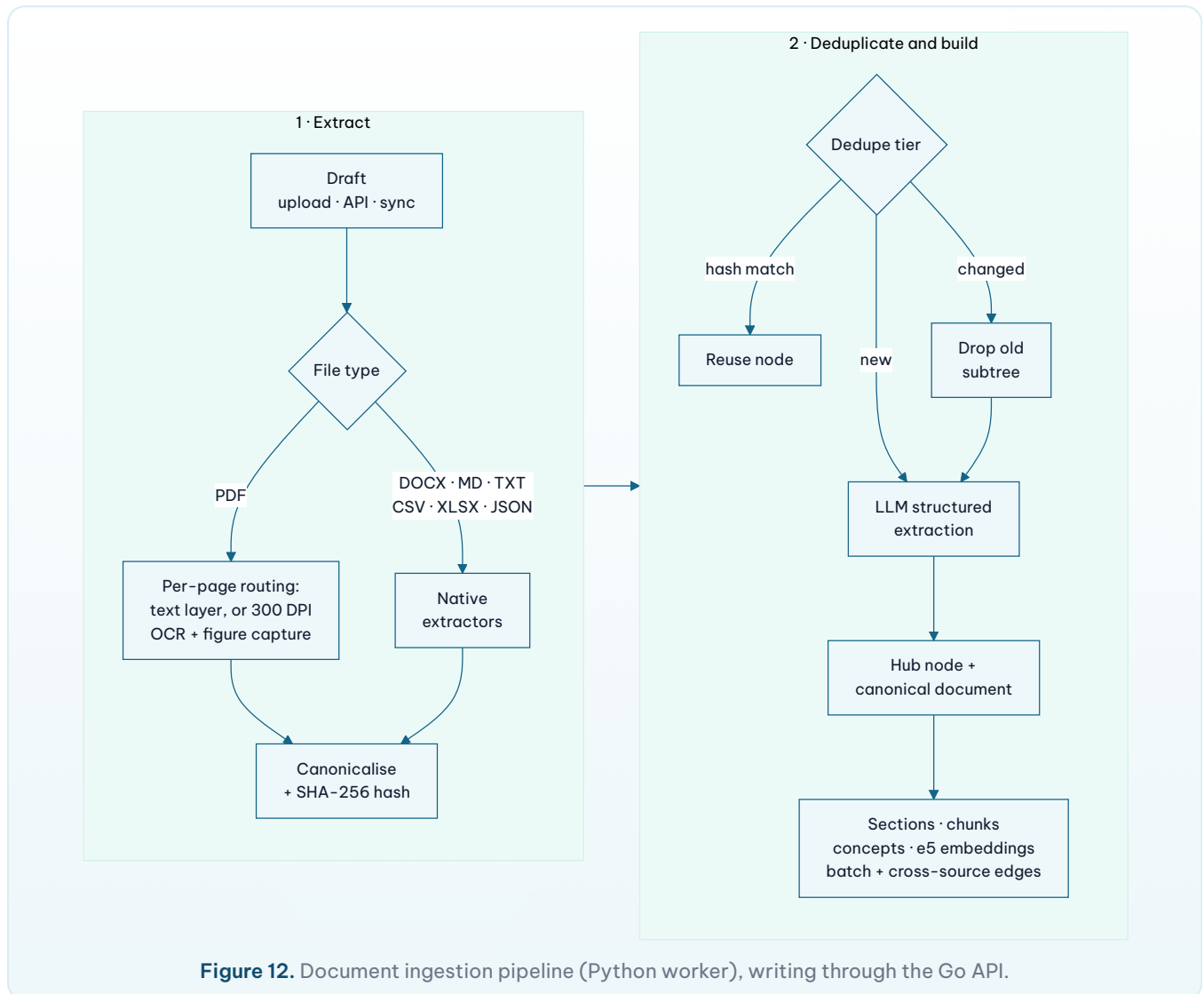
Knowledge enters the graph by three routes: documents (files and structured drafts), live databases (schema catalogues) and source code (symbols). All three write through the Go API, so every node passes the same validation gates.

6.1 Documents LIVE

Entry points. Dashboard drag-and-drop upload; an MCP client uploading a file out-of-band to the bridge's file endpoint; the public ingest API (single draft or batches of up to 50); and a document-management-system sync, configured as a project source connection, that pages through analysed documents, downloads each through a signed URL, ingests it and discards the temporary copy. Every input becomes a **draft node**. Drafts are processed immediately or held for approval according to project settings.

Format	Extraction	Graph output
PDF	pypdf text layer per page; pages with < 50 characters, or with image content and mangled Vietnamese text, are rendered at 300 DPI and OCR'd with Tesseract (<code>vie</code>). Up to 50 pages per file.	Markdown → sections, chunks, concepts, embeddings
DOCX	python-docx paragraphs	Sections, chunks, concepts, embeddings
Markdown, TXT	Direct read	Sections, chunks, concepts, embeddings
CSV, XLSX, JSON	Parsed to JSON / CSV text; trailing empty rows trimmed. Deterministic balance-sheet and income-statement recognition for Vietnamese Circular 200 layouts with English headers.	Hub node (<code>dataset</code> or <code>document</code>) with summary and structured properties

OCR side channel. On scanned pages, RapidOCR (ONNX) extracts document numbers, dates, areas, amounts and ID numbers with pattern rules, with a Tesseract fallback for missing fields. The recovered figures are stored as structured fields and appended to the document text, so they become searchable. No vision model is involved.



Deduplication (“ingest once”). Content is canonicalised (BOM, line endings, whitespace) and hashed. Three tiers apply: (1) the same source with the same hash reuses the node; (2) the same source with a changed hash regenerates: the old sections, chunks and embeddings are removed and rebuilt on the same hub node; (3) the same hash from any other source reuses that node. A lookup error fails the draft rather than risk a silent duplicate.

Structure and embeddings. A Markdown heading tree yields up to 200 `document_section` nodes per document. A section of up to about 1,500 characters becomes a single `document_chunk`; larger sections are packed by paragraph into chunks of at most 1,200 characters, splitting on sentences when needed. Each chunk carries its line and character offsets, so answers can cite the exact passage. Entities from the extraction step become project-scoped `concept` nodes, deduplicated on a diacritic-folded key that also normalises Vietnamese legal-form abbreviations. Reusing concepts across documents is what connects documents to each other. Sections and chunks are embedded with multilingual-e5-small (384 dimensions, L2-normalised, `passage:` prefix) in batches of 32.

Structured extraction. One LLM call per document (temperature 0, routed through the Go AI layer) returns topic, entities, decisions, referenced tables, referenced APIs and people, keeping names in their

original language. It reads the leading 12,000 characters of the document. A second LLM pass proposes cross-source edges (`cross_source_reference` , `references_document` , `references_component`) against up to 80 architecture nodes, and applies those with confidence ≥ 0.5 . Tables named in a document are linked to their schema nodes with `references_table` .

6.2 Live databases LIVE

Connecting a database turns its schema into a navigable part of the graph. Both extraction and graph generation run inside the Go server, extraction on the in-process job engine (3 workers, 3 retries) and generation as a background job, with live progress streamed to the dashboard.

- **Engines:** PostgreSQL (`lib/pq`), MySQL and MariaDB (`go-sql-driver/mysql`), Microsoft SQL Server (`go-mssqldb`), each with its own catalogue querier.
- **Explicit relationships** come from declared foreign keys; junction tables are detected and turned into many-to-many edges.
- **Implicit relationships** are proposed from naming conventions and compatible column types, scored 0–1 by an LLM, and stored with confidence and reasoning. People can confirm, reject or reverse a rejection in the UI.
- **Re-sync** refreshes the catalogue, and every successful sync re-validates approved write playbooks against the new schema (Chapter 9).
- The resulting `schema_table` nodes are also what the query engines use to narrow a question to relevant tables (Chapter 8), and what documents link to.

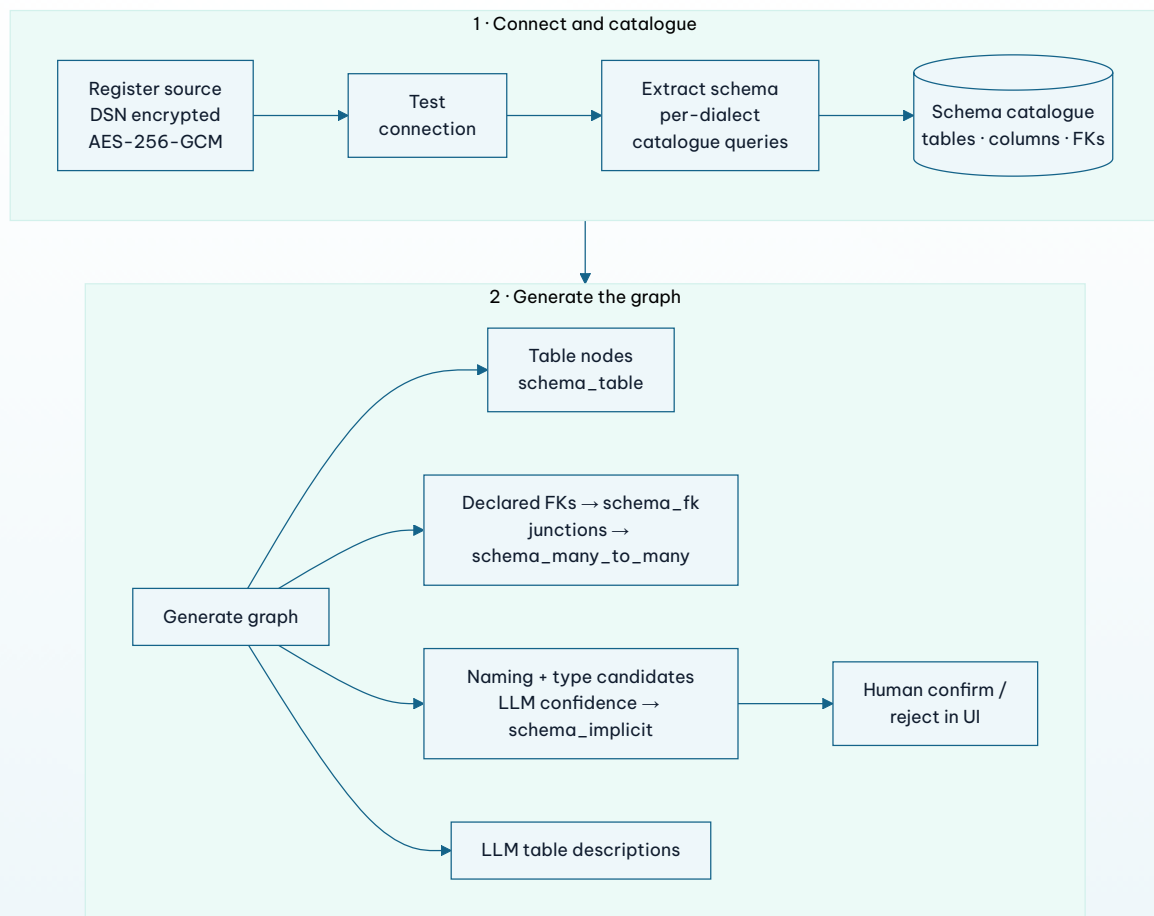


Figure 13. From a live database to a knowledge graph of its schema.

6.3 Source code BUILT

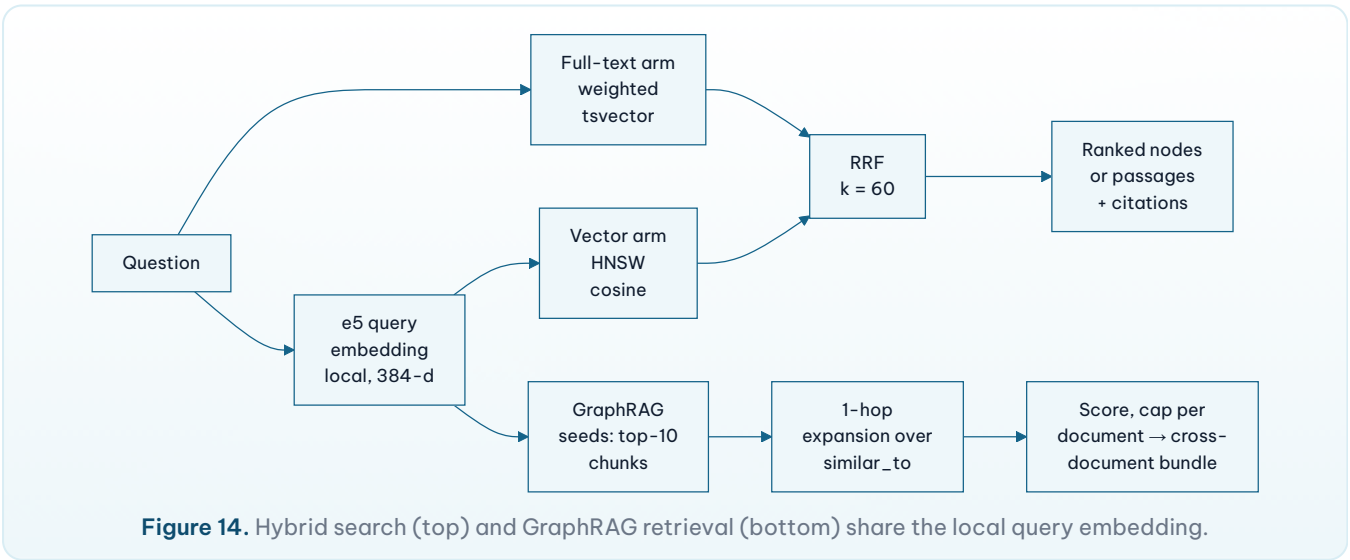
A code indexer (a Python package and CLI, also runnable through the MCP bridge as a local tool on the agent's host) parses repositories with tree-sitter and stores each symbol as an `architecture` node carrying file path, line range, signature and a body hash.

Language	Symbols extracted
TypeScript / JavaScript	functions, classes, methods, interfaces, type aliases, exports, React components, Next.js routes
Python	functions, methods, classes, FastAPI routes, decorators, docstrings
Go	functions, methods (with receiver), structs, interfaces, type aliases, constants, variables
Dart	classes, mixins, enums, extensions, typedefs, functions, members
Markdown	document hubs and heading sections

Re-indexing is incremental: symbols are keyed on file, name and kind, and the body hash detects change, so only new, changed or removed symbols touch the graph. The scanner honours `.gitignore`. Today the indexer records containment (class → method, document → section); import and call-graph edges are not yet produced.

7 Retrieval: search, hybrid ranking & GraphRAG

Retrieval runs in PostgreSQL: lexical and vector search use the same tables and the same transaction boundary as the graph, and a fusion step combines them. Graph structure then widens the result to related passages in other documents.



Search modes

Mode	Mechanism
Full-text (default)	Trigger-maintained weighted <code>tsvector</code> (title/name weighted highest); query terms combined with OR semantics; normalised <code>ts_rank</code> 0-1. Conversation history has its own accent-insensitive (<code>unaccent</code>) index suited to Vietnamese.

Mode	Mechanism
Semantic	The question is embedded by the local e5 model (<code>query: prefix</code>) and matched by cosine distance on an HNSW index over node embeddings; default scope is sections, documents and concepts.
Hybrid	Full-text and semantic arms run concurrently and are fused with Reciprocal Rank Fusion ($k = 60$), de-duplicated by node and tie-broken by recency. It degrades gracefully: if embedding fails it returns full-text only; if one arm fails the other still answers.
Passage search	Hybrid search restricted to <code>document_chunk</code> ; returns the verbatim passage with its section and line range so an agent can quote and cite it.
Structured query	JSON filters over node type, status and properties, with optional multi-hop expansion.

■ Graph navigation

- **Neighbours:** one hop in either direction, paged. **Traverse:** recursive SQL common-table expression up to three hops, filterable by edge type.
- **Document navigation:** document metadata for citation, section outline, section content, parent/child/sibling sections, and backlinks from other sources.
- **Related documents:** documents ranked by the canonical entities they share, weighting rarer entities more heavily; plus the list of entities two documents share.

■ GraphRAG

Cross-document retrieval rests on `similar_to` edges between chunks of *different* documents. A background link worker runs every five minutes over recently ingested projects (or on demand), finds each chunk's nearest cross-document neighbours, keeps up to 5 per chunk above a cosine similarity of 0.90, and writes them as edges. At query time the retriever seeds with the 10 most relevant chunks (using a raised HNSW `ef_search` of 400 so the chunk subset is not starved by post-filtering), expands one hop across `similar_to` , scores each neighbour by seed relevance × edge similarity, caps results per document, and can enrich the bundle with parent sections, shared-entity expansion and inline snippets.

■ Vector indexes

Column	Dimensions	Index	Model	Used for
<code>knowledge_node_embeddings</code>	384	HNSW, cosine	multilingual-e5-small (local)	Semantic & hybrid search, passages, GraphRAG, entity blocking
<code>agent_context_embeddings</code>	384	HNSW, cosine	multilingual-e5-small (local)	Agent memory recall
<code>table_embeddings</code>	1,536	IVFFlat, cosine	Hosted embedding API (optional)	Table selection for data queries on large schemas BUILT

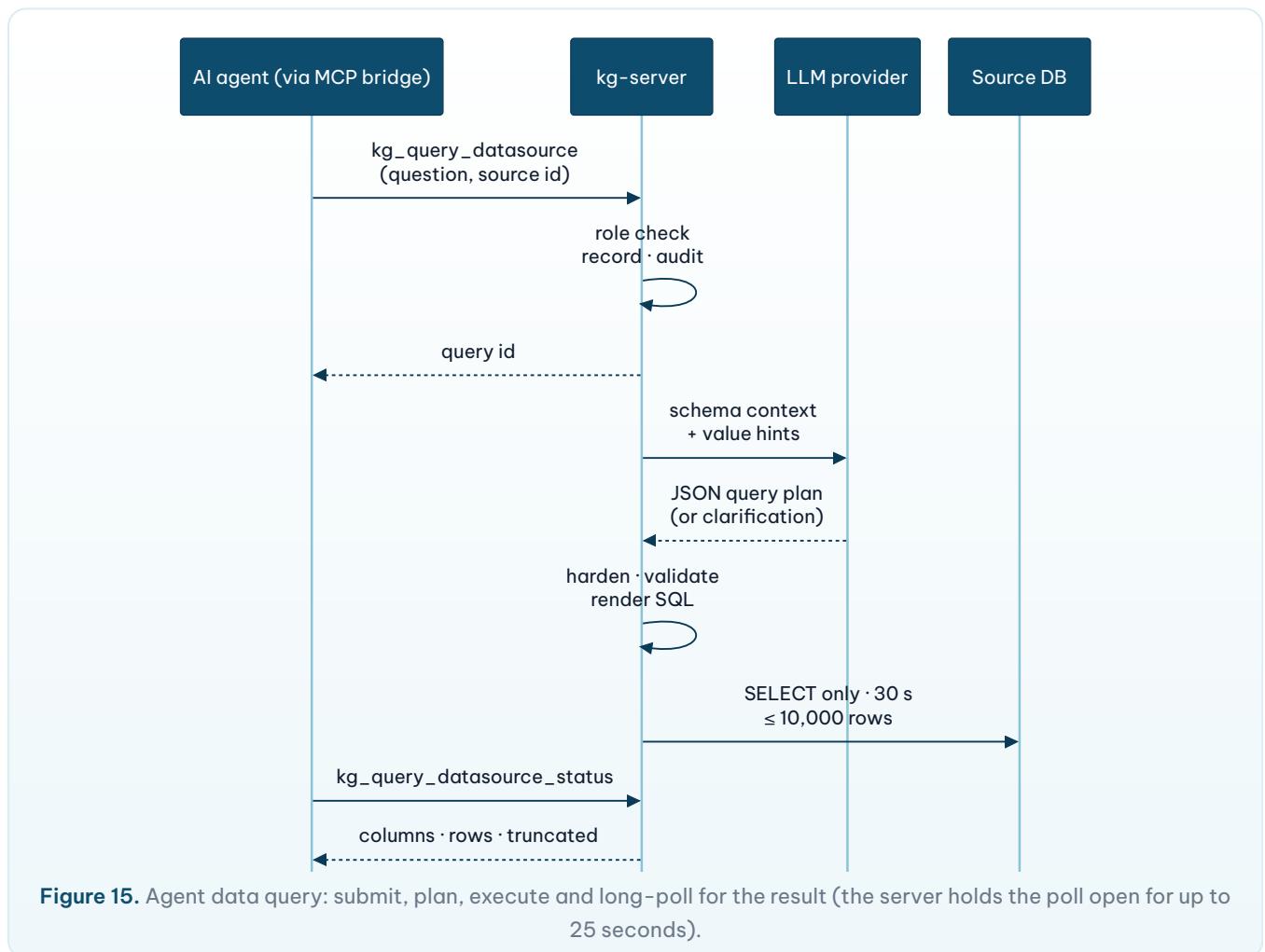
8 Natural-language data query

Agents ask questions of live databases in plain language. On this agent / API path the LLM never writes the SQL that runs: it produces a structured **query plan** which the server hardens,

validates against the extracted schema and renders into parameterised, read-only SQL. The dashboard's chat modes are described at the end of the chapter.

■ Pipeline stages

1. **Plan cache.** Plans are cached on the normalised question plus a fingerprint of the schema and prompt, so repeated questions skip the LLM and a schema change invalidates stale plans.
2. **Context.** The planner builds a schema context from the extracted catalogue, adding categorical *value hints* sampled read-only from the source so filters use real literal values. When table embeddings are configured, the most relevant tables are selected by vector similarity.
3. **Plan.** The LLM (chosen by the provider selector with failover) returns a JSON plan (tables, joins, filters, aggregations, grouping, having, ordering, limit) or asks for clarification.
4. **Harden and repair.** Deterministic passes repair aliases, promote stray tables to joins, de-duplicate and validate join conditions, check every column against the extracted schema, cap ranked queries at 100 rows and correct ranking direction. A failed execution gets one simplification retry.
5. **Render.** The SQL generator emits dialect-specific SQL with every value bound as a parameter; operators come from an allow-list and column, interval and sub-query shapes are accepted only through closed grammars.
6. **Execute and store.** Results, status and the generated SQL are stored with the query record for the caller to fetch and for audit.



Execution guard (agent path)	Setting
Statement shape	Must begin with <code>SELECT</code> or <code>WITH</code>
Session mode	PostgreSQL sources: session set to <code>READ ONLY</code>
Connection	Fresh single-connection pool per query; credentials decrypted at use
Timeout	30 seconds per query
Result cap	10,000 rows, with a <code>truncated</code> flag; the caller can limit the rows it receives with <code>max_rows</code>
Values	Always bound parameters, never string-concatenated
Record	Every submission stored in <code>ai_queries</code> and written to the audit trail

Clarification instead of guessing

When a question is ambiguous (for example, several candidate metrics or date columns), the planner returns `clarification_needed` with the question to put to the user, rather than running a plausible but wrong query. The status is stored, so the agent can ask the user and resubmit.

■ Dashboard conversational analysis

The dashboard's chat uses two streaming engines in the Python indexer. The Go server proxies the Server-Sent Events stream (at most 3 concurrent chat streams per user), passing per-request provider and data-source context: the provider key and the encrypted connection string. The indexer decrypts the connection string and connects to the source database directly. These engines rely on their own SQL checks and row caps; they do not set a database-level read-only session (adding one is a hardening item, Document 03, Chapter 8).

Engine	How it works
Fast path	Narrows the schema using the graph's <code>schema_table</code> nodes and question keywords; the LLM fills a structured query-plan tool; SQL is rendered deterministically with bound filter values and a row limit; then the results are summarised, a presentation format is chosen (text, table or chart), and insights and suggested follow-ups are streamed as typed blocks.
Agentic path	A tool-using loop with <i>quick</i> (3 iterations, 30k tokens) and <i>deep</i> (12 iterations, 100k tokens) tiers. Tools: graph search, table schema, SQL execution, charting and clarification; the deep tier adds neighbours, traversal, document structure and semantic search. Here the model writes the <code>SELECT</code> statement itself; it must pass a read-only validator (read statements only, single statement, blocked write/DDL keywords, length cap), and results are trimmed to 50 rows; a loop guard blocks repeated calls. A clarification pauses the agent, saves its state for 10 minutes, and resumes it exactly once.

9 Controlled writes LIVE

Writing to a customer database is off by default and, when enabled, runs through two narrow paths: a validated single-row insert, and named multi-step **playbooks** that must be approved before they can be executed. Every attempt is audited. The write path has so far been exercised at small volume.

Single-row insert

1. **Opt-in per data source:** `allow_writes` defaults to false and is set by an administrator.
2. **Explicit table whitelist:** an empty whitelist means nothing is writable.
3. **Validation against the extracted schema:** required columns present, no unknown columns, type and length checks.
4. **Dedicated write connection** with a 30-second timeout, used once.
5. **Audit of every attempt:** successful, rejected and failed attempts all land in `source_write_audit`.

Agents discover what they may write with `kg_describe_table` (the column specification) before calling `kg_insert_datasource_row`.

Write playbooks

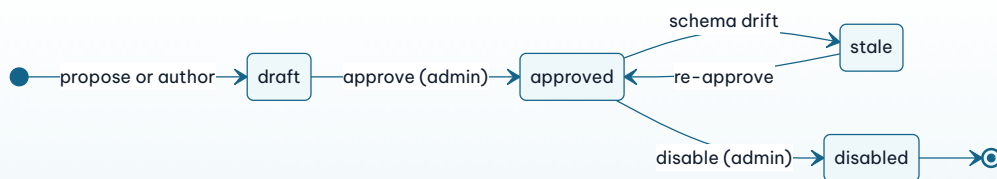


Figure 16. Playbook lifecycle. Only approved playbooks can execute (developer role; all steps in one transaction); schema drift automatically takes a playbook out of service.

- A playbook is a named, versioned JSON definition of inputs and steps, using a small template vocabulary (`$input.*`, `$item.*` with `foreach`, generated IDs, `$now`). It can express, for example, a header row with its detail lines.
- **Proposal is deterministic:** the server derives a draft from header/detail table relationships in the schema, with no LLM involved.
- **Execution is atomic:** all steps run inside one database transaction and roll back together on any failure.
- **Drift protection:** after every successful schema sync, playbooks are re-validated and any that no longer match are marked `stale`.
- **Roles:** for user-linked keys, approve, disable and delete require project admin and execute requires developer. Extending these role checks to keys not linked to a user is a hardening item before production (Document 03, Chapter 8).

Write paths currently support PostgreSQL sources. Both MCP write tools carry no read-only hint, so clients can ask the user to confirm, and read-only bridge tokens cannot call them.

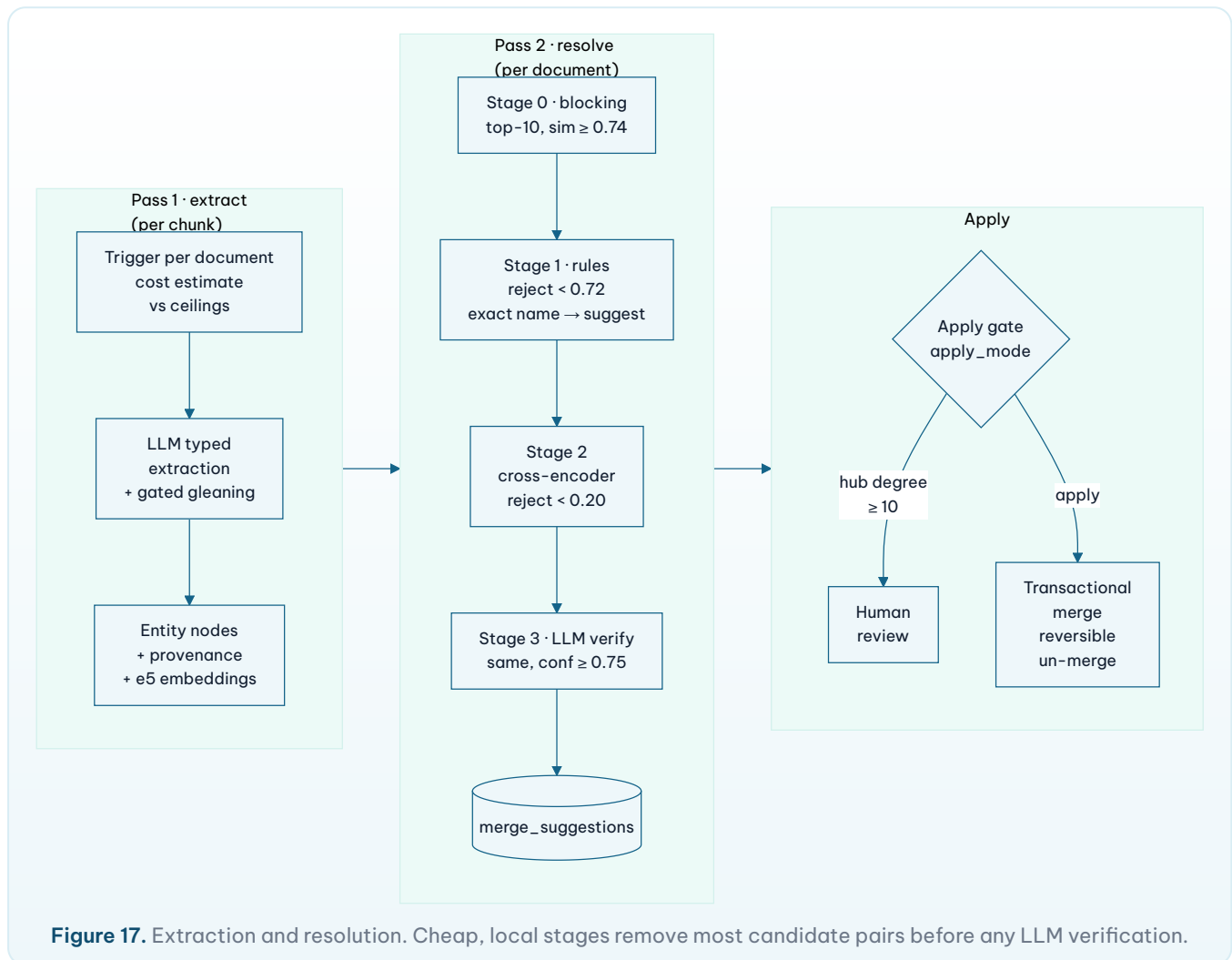
Writes into the graph follow the same review discipline: agents submit knowledge as **drafts** (`kg_ingest_node`, or `kg_ingest_batch` for up to 50), which are approved singly or in bulk and then processed into nodes.

10 Entity resolution & master records LIVE

Documents mention the same person, company or place in many spellings. A two-pass pipeline extracts typed entities per chunk and then decides, through progressively more expensive stages, which ones are the same, recording every decision before anything is merged.

■ Pass 1: extraction

Extraction is started per document through the API. The server first estimates the cost (chunks × average tokens × model rates × gleaning rounds) and refuses runs that would exceed the configured ceilings (\$5 per run, \$1 per document); chunks already extracted at the same content hash are skipped. Each chunk is then queued as its own job. The LLM returns entities of eight types (person, organisation, concept, event, location, artifact, document reference, project) and relations from a closed vocabulary (`works_for`, `part_of`, `mentions`, `causes`, `related_to`, `derived_from`, `evidence`), keeping names in the original language. A *gleaning* pass (at most 2 rounds) runs only on long, entity-sparse chunks and stops as soon as a round finds nothing new. Every entity keeps its provenance: source document, chunk and sentence span.



■ Pass 2: resolution stages

Stage	Rule	Cost
0 · Blocking	Embed each entity (name + description); retrieve the 10 nearest neighbours above cosine similarity 0.74.	Local
1 · Rules	Reject below 0.72. Suggest a merge (confidence 0.99) on an exact match after normalisation (Unicode NFC, case, separators, honorifics removed). No other rule-based merges.	Local
2 · Cross-encoder	Local <code>bge-reranker-v2-m3</code> scores the pair; below 0.20 is rejected. It never approves a merge on its own.	Local, no API cost

Stage	Rule	Cost
3 · LLM verify	Pairs batched six at a time (concurrency 10); a suggestion is written only when the model judges them the same entity with confidence ≥ 0.75 .	LLM
Optional · adjudication	When configured, a second LLM reviews fuzzy (non-exact-name) suggestions in batches of 20 with Vietnamese administrative-entity rules and moves doubtful ones to review.	LLM

■ Applying merges

- Suggestions are applied only when the server's `apply_mode` is `apply`; it is currently configured on. The check is a literal comparison, so any other value fails closed.
- **Hub guard:** a pair where either node has 10 or more connections is never auto-merged; it goes to human review.
- **Merge is one transaction:** follow the `merged_into` chain to the live canonical, union aliases and provenance, re-point edges without loss, supersede the member node, append a resolution-audit entry and write with optimistic locking. Every change is stamped with a merge-operation ID, so **un-merge** restores the exact prior state.

Measured quality (internal benchmark)

Before auto-apply was switched on, the deployed three-stage pipeline was gated against a small internal Vietnamese benchmark (30 entity groups, 86 duplicate pairs; measured June 2026).

Results: merge precision **1.000** and recall **0.915**, with zero false positives. This is a small, domain-specific benchmark and should not be read as production accuracy on other corpora.

■ Master and derived records

The surviving canonical node of a merge acts as the **master record** for that entity (it keeps its entity type): it carries the unioned aliases, full provenance back to every source sentence, and an audit history of each merge. Separately, external systems can write **derived records** (summaries or reference records they own) back into the graph idempotently, link them to their evidence documents, fetch the latest one (`kg_get_master_record`) and revoke them.

11 Agent memory BUILT

Agents need to remember preferences, decisions, facts and corrections across sessions. DAAB provides a small, durable memory store with hybrid recall, separate from the main graph but in the same database.

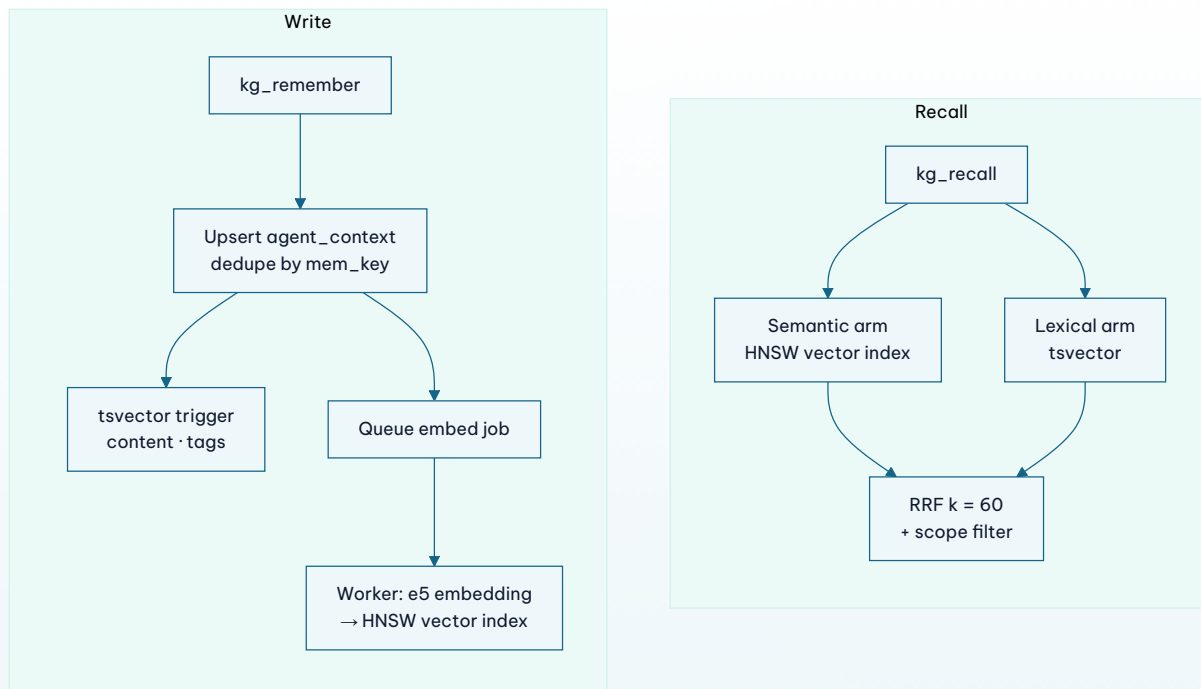


Figure 18. Memory write (embed-on-write via the job queue) and hybrid recall.

- **Kinds:** preference, decision, fact, correction. **Scopes:** project, user, agent. Memories under user scope are returned only to their owner.
- **Idempotent writes:** an optional `mem_key` (unique per project, user and scope) turns a write into an update, so an agent can refine a memory instead of piling up near-duplicates.
- **Recall** fuses semantic and lexical results with the same Reciprocal Rank Fusion used by search.
- **Upkeep:** a background retention worker de-duplicates and bounds growth, and a touch writer records when each memory was last recalled.
- **Conversation history:** separately, agents can full-text search past conversation threads (`kg_search_sessions`), with each read recorded in an access log.

12 MCP bridge LIVE

The bridge is DAAB's integration surface for AI agents. It advertises 55 tools over the Model Context Protocol, 53 of them wired to the API (two have no server route yet), and maps each tool call onto the REST API, so agents get exactly the validation, authorisation and audit that the dashboard gets.

• Transports and deployment

- **stdio** (default): a single local client launches `kg-bridge serve` as a subprocess.

- **Streamable HTTP** (`serve --http`): a shared network endpoint built on the official MCP Go SDK, with a per-session server instance. Health and readiness probes sit outside authentication. The bridge ships in the same image as the API server.

Authentication modes

Mode	Behaviour
Passthrough (multi-tenant)	Each request's bearer token is the caller's own DAAB API key. The bridge builds a per-session API client with that key, so the server enforces that key's role and project scope. One bridge serves many consumers, each scoped to the projects its key grants (see Chapter 3 for how project scope is enforced). Keys are issued in the dashboard with a role and a project scope.
Gate token (single-tenant)	Callers present a bridge token (at least 32 bytes, or 43 characters, long, checked at start-up); an optional second token grants a read-only scope under which every write-class tool and local-execution tool is refused. The bridge calls the API with its own service key.

The HTTP middleware chain is: panic recovery → 60-second request timeout → authentication → rate limit (120 calls per 60 seconds per identity) → call log → MCP dispatch. Write-class tools are published without the MCP `readOnlyHint` annotation, so compliant clients ask the user before invoking them. An optional start-up switch makes the bridge refuse any non-loopback address, for deployments that require TLS to be terminated by a fronting proxy.

Per-call pipeline

1. **Scope check**: under a read-only token, write-class and local-execution tools are refused.
2. **Misroute recovery**: calls that clearly belong to another tool (for example, a natural-language question with a data-source ID sent to the structured-query tool) are rerouted, or answered with a corrective error that tells the model which tool to use.
3. **Project pinning**: when the bridge is configured with a default project, it overwrites any project ID the model supplies (with a log entry). This protects against a model mis-addressing a call; it is not the tenancy boundary. Access is governed by the API key on the call, as enforced by the API (Chapter 3).
4. **Schema validation** at the bridge (UUID patterns, enums, bounds), then mapping to the HTTP method, path, query and body; the result is rendered as MCP text content.

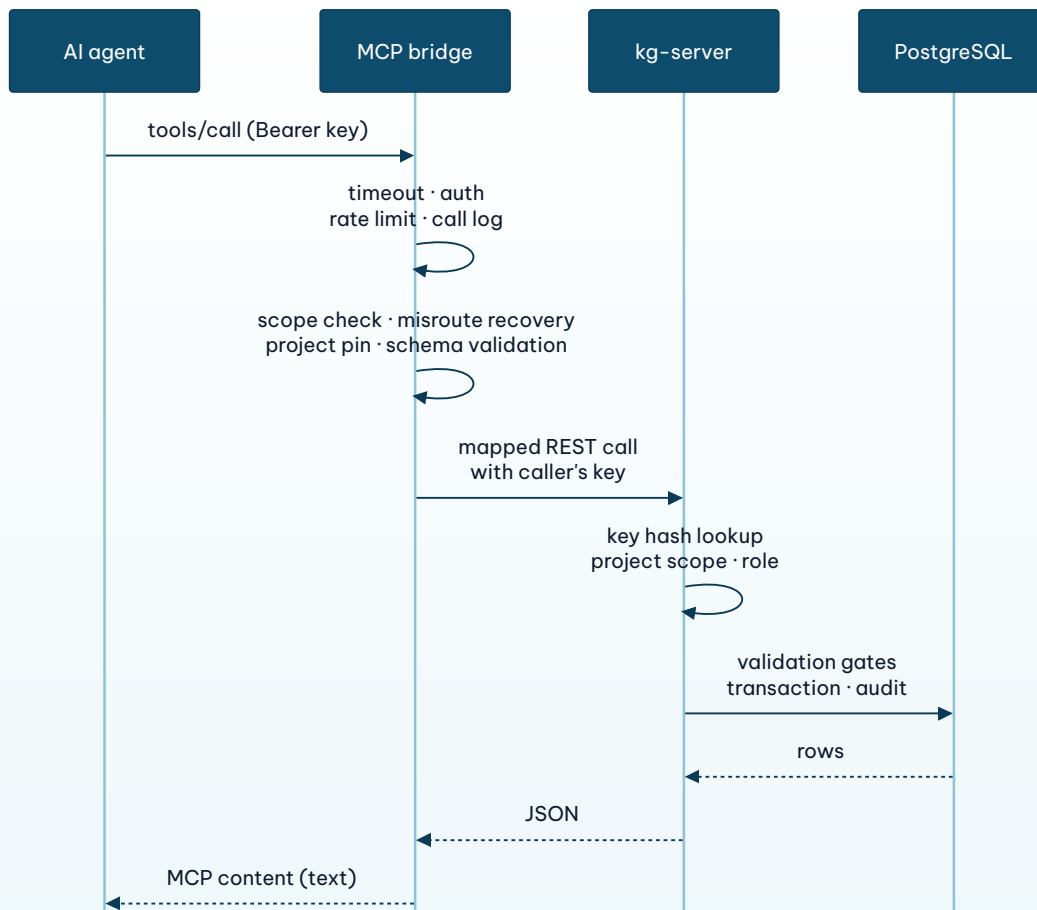


Figure 19. One MCP tool call end to end (Streamable HTTP, passthrough mode).

Connection context. For each caller, the bridge looks up the projects and data sources that caller can reach (cached for 5 minutes) and returns them as MCP server instructions and in the descriptions of the key tools. An agent therefore starts with the IDs it needs and does not spend calls discovering them.

Files stay out of the protocol. `kg_request_file_upload` returns an upload URL; the client sends the bytes over plain HTTP to the bridge, which forwards them, under the bridge's own service key, to the upload endpoint of the bridge's default project. Large documents never travel as MCP messages.

Local tools. `kg_index_source` / `kg_index_status` run the code indexer on the agent's host, where the bridge runs, against a local path and report progress. They work only where the Python indexer CLI is installed on that host; the bridge container image does not include it. Together with the upload-URL tool, they are the only three tools not backed by a REST route.

Tool catalogue. The 53 wired tools fall into nine groups: knowledge capture (15), search and navigation (9), documents (6), ingestion (7), live data (4), controlled writes (3), memory and sessions (5), records (2) and local code indexing (2); 25 are read-class, 25 write-class and 3 run locally. Appendix A lists every tool with its purpose.

13 Web dashboard LIVE

A Next.js 16 / React 19 application for the people side of the platform: exploring the graph, analysing data conversationally, connecting sources, reviewing ingestion and administering access and AI providers. It runs as a backend-for-frontend (BFF): the browser only ever talks to the dashboard's own origin.

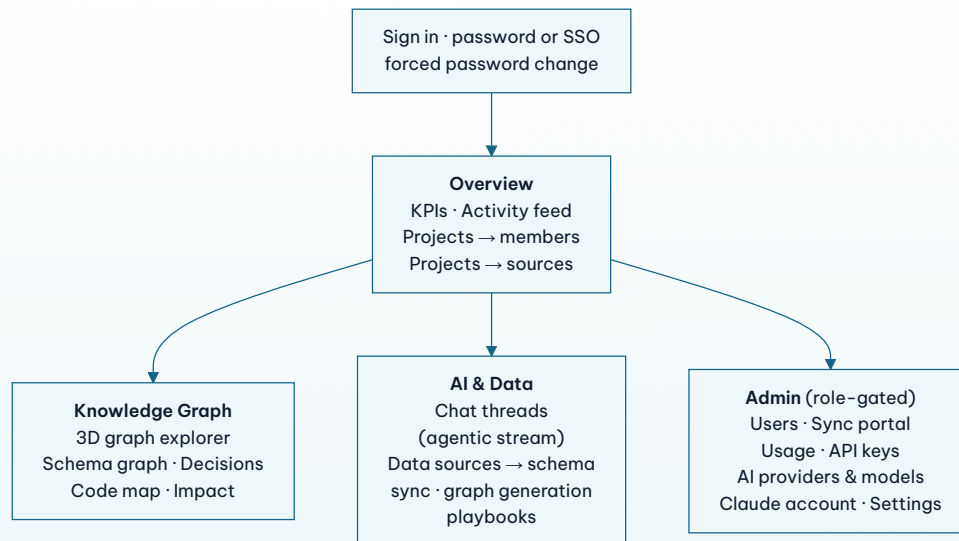


Figure 20. Dashboard page map, grouped as in the navigation sidebar (33 page routes in total, including sign-in, drill-ins and pages outside the sidebar).

■ BFF proxy and sessions

- **Sign-in** runs as a server action: the dashboard calls the API's login (or SSO exchange) endpoint, receives a per-user API key and stores it in an encrypted, HTTP-only session cookie. A new login revokes the user's previous dashboard key, so each user has a single active dashboard session.
- **Proxy:** a catch-all route maps `/api/kg/*` to the API's `/api/v1/*`, attaching the session's key as a bearer token. Server-Sent Events pass through unbuffered; graph responses are normalised into a single nodes/edges shape; an upstream 401 ends the session. A second route streams multipart uploads.
- **Authorisation** is enforced in the Go API; the dashboard only hides the Admin section from non-admins. Feature flags come from the API's public settings.
- **Data fetching:** server state is managed with TanStack Query (33 hook modules).

■ Real-time

Mechanism	Used by	Detail
Streaming fetch (SSE over POST)	Chat	Typed events: progress, content tokens, rich blocks, tool-call start/end, reasoning, clarification requests, graph-node references, data results, suggested actions, done. Retries, inter-event timeout and a connection cap on the client.
EventSource (SSE over GET)	Schema sync and graph generation	Phase-by-phase progress (extracting → generating nodes → mapping edges → detecting implicit relations → done).
Adaptive polling	Ingestion, document-system sync, usage, drafts	Polls only while a job is in a non-terminal state and stops when it completes.

■ Graph visualisation

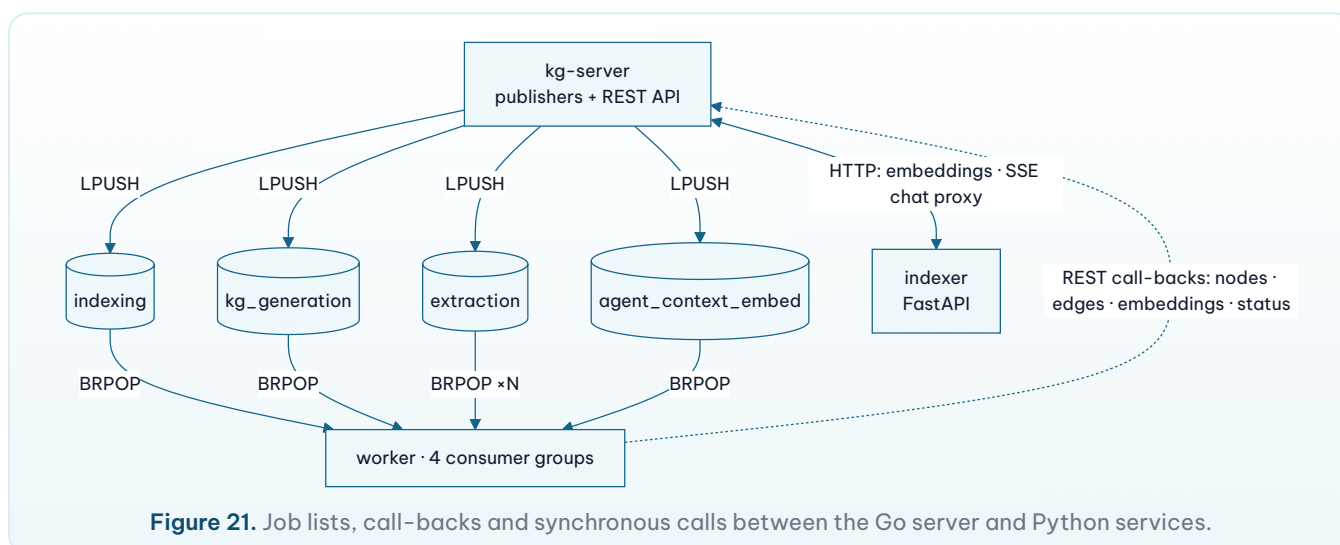
- **3D knowledge-graph explorer:** a React Three Fiber scene that draws all nodes as one instanced mesh and all edges as one line set (one draw call each), with bloom post-processing and orbit

controls. Force layout (force, anchor and cluster modes) runs in a Web Worker with d3-force-3d and freezes when it converges, keeping the UI thread free; the view is designed for graphs of thousands of nodes.

- **Schema graph:** an entity-relationship view of any connected database (tables, foreign keys, many-to-many and implicit relations) rendered with Cytoscape and a dagre layout.
- **Impact view:** a 3D force graph of a chosen node and its immediate neighbours.

14 Inter-service communication

Services communicate in three ways: asynchronous jobs on Redis lists, synchronous HTTP between the Go server and the Python indexer, and HTTP call-backs from workers into the Go API, through which every graph write passes.



List	Message types	Purpose
indexing	index_project , index_changed	Full or incremental code indexing of a repository
kg_generation	kg_generation , extract_upload , document-system sync	Approved document drafts into the graph; uploaded-file extraction; pulling documents from a connected document-management system
extraction	extract_document (one chunk), resolve_document	Chunk-level entity extraction, then per-document resolution; consumed by several concurrent coroutines
agent_context_embed	embed_agent_context	Embed-on-write for agent memories

■ Synchronous calls

- **Go → indexer:** query embeddings for semantic and hybrid search, GraphRAG and memory recall (10-second timeout); streaming chat, proxied as SSE with at most 3 concurrent chat streams per user.
- **Workers → Go:** all graph writes and status reports use authenticated REST calls: draft completion, content and properties; node and memory embedding batches; canonical-document records; extraction progress (chunk complete, document resolved); resolution candidates, suggestions, merges and un-merges; GraphRAG link requests; derived records. The worker-facing endpoints sit under the same API-key authentication as the rest of the `/api/v1` API.

- **Go** → **browser**: progress streams for schema sync, graph generation and document-system sync, fed by an in-process broadcaster.

■ Delivery semantics

Lists are FIFO, and a pop removes the message, so delivery is at most once. Resilience is built into the jobs: each handler isolates failures per item (per draft, per document, per file), reports failures back to the API so they are visible in the dashboard, and never marks a failed item complete. Server-side, a heartbeat monitor fails schema-sync jobs that have been idle for more than 30 minutes, so nothing appears to run forever. A recovery sweep that re-publishes stalled extraction runs is implemented but not yet scheduled **PLANNED**. Workers shut down gracefully on SIGTERM.

A Appendix A – MCP tool catalogue

R = read-class (published with `readOnlyHint`, allowed under read-only tokens); W = write-class; L = local execution on the agent's host, where the bridge runs. Two further tools (a scoped context summary and an impact analysis) are already advertised to clients but have no server route yet, so calls to them fail **PLANNED**; they are not counted. The two L-class indexing tools need the code indexer installed on the agent's host, where the bridge runs.

Tool	Class	Purpose
kg_store_decision	W	Create a decision node (rationale, alternatives), optionally with edges
kg_store_concept	W	Create a concept node
kg_store_requirement	W	Create a requirement node
kg_store_task	W	Create a task node
kg_store_architecture	W	Create an architecture node
kg_store_discovery	W	Create a discovery node
kg_update_decision ... kg_update_discovery (6)	W	Typed partial update of each of the six node types above
kg_update	W	Generic partial update with optimistic locking
kg_link	W	Create a whitelist-validated edge
kg_deprecate	W	Deprecate a node (the node API has no delete)
kg_search	R	Full-text, semantic or hybrid node search
kg_search_chunks	R	Passage search returning verbatim text with section/line citation
kg_query	R	Structured query by node type, filters and optional hops
kg_get_node	R	Fetch one node
kg_get_history	R	Version history of a node
kg_get_neighbors	R	One-hop neighbours
kg_traverse	R	Multi-hop traversal (up to 3 hops)
kg_graph_retrieve	R	GraphRAG cross-document chunk bundle
kg_list_projects	R	Projects accessible to the caller
kg_get_document	R	Document citation metadata
kg_get_document_outline	R	Section tree of a document
kg_get_section_neighbors	R	Parent, children and siblings of a section
kg_get_backlinks	R	References to a node from other sources
kg_related_documents	R	Documents sharing canonical entities, ranked by specificity
kg_document_shared_entities	R	Entities two documents have in common
kg_ingest_node	W	Submit one knowledge draft

Tool	Class	Purpose
kg_ingest_batch	W	Submit up to 50 drafts
kg_list_drafts	R	List drafts and their status
kg_approve_drafts	W	Approve up to 50 drafts
kg_process_drafts	W	Queue approved drafts for graph generation
kg_ingest_status	R	Poll a file ingestion until the document is ready
kg_request_file_upload	L	Get the out-of-band upload URL for a file
kg_list_datasources	R	Discover connected databases
kg_query_datasource	R	Ask a natural-language question of a live database (audited)
kg_query_datasource_status	R	Long-poll for the query result (row count limit via <code>max_rows</code>)
kg_describe_table	R	Column specification for a writable table
kg_insert_datasource_row	W	Opt-in, whitelisted, validated single-row insert
kg_list_playbooks	R	Approved write operations for a data source
kg_execute_playbook	W	Run an approved multi-step write in one transaction
kg_remember	W	Write a durable, scoped memory
kg_recall	R	Hybrid memory recall
kg_store_session	W	Start an agent work session
kg_end_session	W	End a session with a summary
kg_search_sessions	R	Full-text search of conversation history
kg_upsert_derived_record	W	Idempotently write a record owned by an external system
kg_get_master_record	R	Latest derived record for a project
kg_index_source	L	Index a local code repository into the graph
kg_index_status	L	Poll a local indexing job

B Appendix B – REST API summary

251 routes: 247 registered by the handler package plus 4 on the outer router (liveness, readiness, version and the sync-progress WebSocket). All `/api/v1/*` routes require an API key except login and the SSO exchange. Counts below are exact and sum to 251.

Area	Routes	Representative endpoints
Graph nodes, edges & documents	38	<code>POST /nodes</code> , typed create/update, <code>PATCH /nodes/{id}</code> , <code>/deprecate</code> , <code>/history</code> , <code>/neighbors</code> , <code>/traverse</code> , <code>/outline</code> , <code>/backlinks</code> , <code>POST /edges</code>
Query, search & retrieval	6	<code>/query</code> , <code>/search</code> , <code>/search-chunks</code> , <code>/retrieve/graph</code> , <code>/schema-graph</code>
Ingestion: drafts, uploads, connections, canonical documents, derived records	39	<code>/projects/{id}/ingest[/batch /upload]</code> , <code>/draft-nodes/*</code> , <code>/uploads/*</code> , <code>/connections/*</code> , <code>/canonical-documents/*</code> , <code>/derived-records</code>
Extraction & entity resolution (worker-facing)	17	<code>/ingestion/documents/{id}/extract</code> , <code>/internal/extraction/*</code> , <code>/internal/resolution/*</code> , <code>/internal/graphrag/link</code> , <code>node-embedding batches</code>
Data sources, schema sync & graph generation (incl. progress streams)	28	<code>/data-sources/*</code> , <code>/test-connection</code> , <code>/extract-schema</code> , <code>/sync-schema</code> , <code>/generate-kg</code> , <code>/kg-edges/{id}/confirm reject</code> , <code>/sync/{job}/*</code>

Area	Routes	Representative endpoints
Natural-language query, chat threads & favourites	25	<code>/ai-queries</code> , <code>/ai-query/stream</code> , <code>/ai/clarification</code> , <code>/threads/*</code> , <code>/favorites/*</code>
Source-database writes & playbooks	11	<code>/tables/{t}/describe</code> , <code>/insert-row</code> , <code>/playbooks/*</code> (propose, approve, disable, execute)
Sessions & agent memory	10	<code>/sessions/*</code> , <code>/sessions/search</code> , <code>/agent-context/remember recall</code>
Projects & members	12	<code>/projects/*</code> , <code>archive/unarchive</code> , <code>stats</code> , <code>/index</code> , <code>/members/*</code>
Users, authentication & API keys	25	<code>/users/*</code> , <code>/auth/login supabase logout change-password</code> , <code>provider OAuth</code> , <code>/api-keys/*</code>
AI providers, models & budgets	13	<code>/ai-providers/*</code> (incl. health check, models), <code>/ai-models/*</code> , <code>/ai-functions</code> , <code>/ai-budget-stats</code>
Settings, audit, activity, usage & queues	14	<code>/settings/*</code> , <code>/audit/*</code> , <code>/activity/*</code> , <code>/usage/dashboard</code> , <code>/queue/*</code> , <code>/rate-limit/status</code>
Benchmarks BUILT	10	<code>/benchmark/questions runs results</code> (not exposed in dashboard navigation)
Health & version	3	<code>/healthz</code> , <code>/readyz</code> (database ping), <code>/api/v1/version</code>
Total	251	

Selected source locations (repository-relative): API composition root `cmd/kg-server/main.go` ; MCP tool routing `internal/bridge/client.go` ; hybrid fusion `internal/store/rrf.go` ; query execution guards `internal/service/source_executor.go` ; GraphRAG `internal/service/graph_retriever.go` ; merge `internal/service/merge.go` ; document pipeline `src/ennam_kg/ingestion/pipeline/` ; resolution `src/ennam_kg/resolution/pass2.py` ; BFF proxy `src/app/api/kg/[...path]/route.ts` .